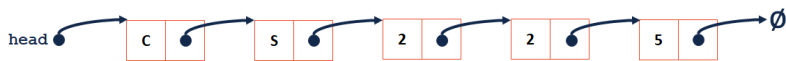


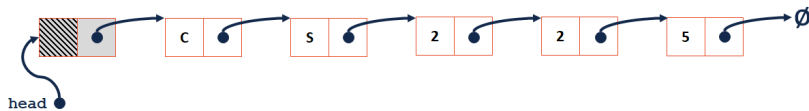
Finding in a list:

List.cpp	
1	#include "List.h"
2	
3	ListNode *& List::find_(unsigned index) const {
4	
5	
6	
7	
8	
9	
10	
11	
12	}

1. What is the return type of `_find`?



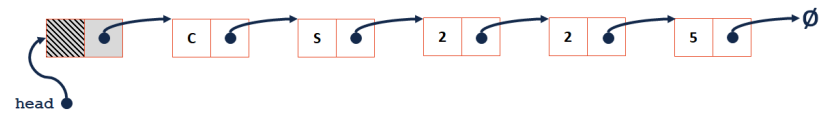
2. The use of a **sentinel head node**:



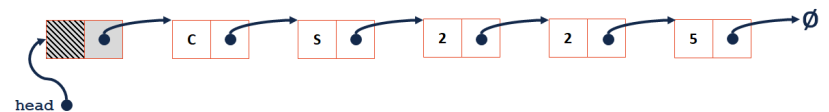
3. Cost of sentinel node?

Class templates:

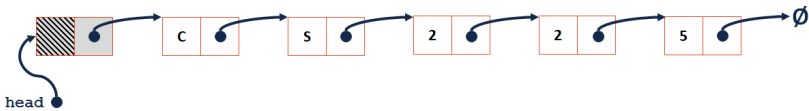
List.cpp - get()	
11	T & List::get(unsigned index) const {
12	
13	
14	
15	
16	}



List.cpp - insert()	
11	void List::insert(T & t, unsigned index) {
12	
13	
14	
15	
16	}



```
List.cpp - remove()
11 T & List::remove(unsigned index) {
12
13
14
15
16 }
```



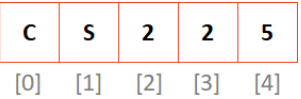
List Implementation #2: _____

```
List.h
1 #ifndef LIST_H
2 #define LIST_H
3
4 template <class T>
5 class List {
6 public:
7     /* ... */
28 private:
29
30
31
32 };
33
34 #endif
```

Implementation Details:

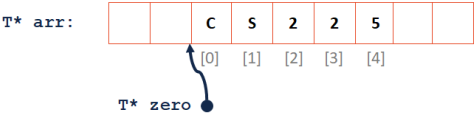


1. What is the running time of `insertFront()`?



2. What is our resize strategy?

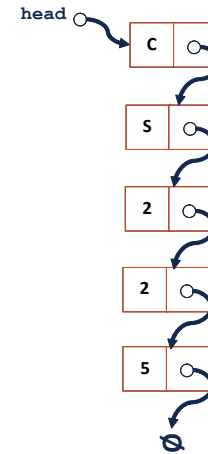
3. What is the running time of `get()`?



	Singly Linked List	Array
Insert/Remove at front		
Insert at given location		
Remove at given location		
Insert at arbitrary location		
Remove at arbitrary location		

Running times of List operations:

	Singly Linked List	Array
Insert/Remove at front		
Insert at given location		
Remove at given location		
Insert at arbitrary location		
Remove at arbitrary location		



Stack Implementation #1: _____

Stack.h	
1	<code>#ifndef STACK_H</code>
2	<code>#define STACK_H</code>
3	
4	<code>template <class T></code>
5	<code>class Stack {</code>
6	<code>public:</code>
7	
8	
9	
10	
11	
12	
13	<code>private:</code>
14	
15	
16	
17	
18	<code>};</code>
19	<code>#endif</code>

Stack.cpp	
1	<code>#include "Stack.h"</code>
2	
3	<code>void Stack::push(T & t) {</code>
4	
5	
6	
7	
8	
9	
10	<code>}</code>
11	
12	<code>T & Stack::pop() {</code>
13	
14	
15	
16	
17	
18	
19	<code>}</code>

4. A stack is a _____ data structure

...that stands for:

Stack Implementation #2: _____

Stack.h

```
1 #ifndef STACK_H
2 #define STACK_H
3
4 template <class T>
5 class Stack {
6     public:
7
8
9
10
11
12
13     private:
14
15
16
17
18 };
19 #endif
```

T* arr:

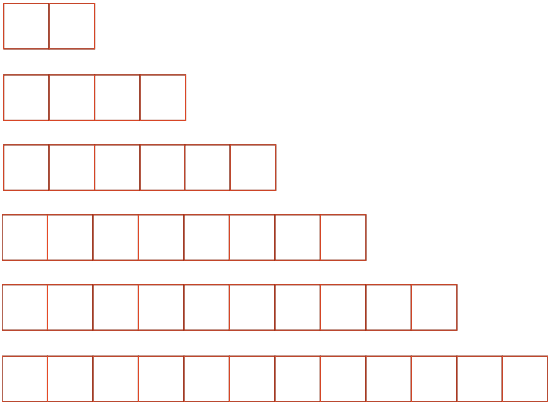
C	S	2	2	5		
[0]	[1]	[2]	[3]	[4]		

Stack.cpp

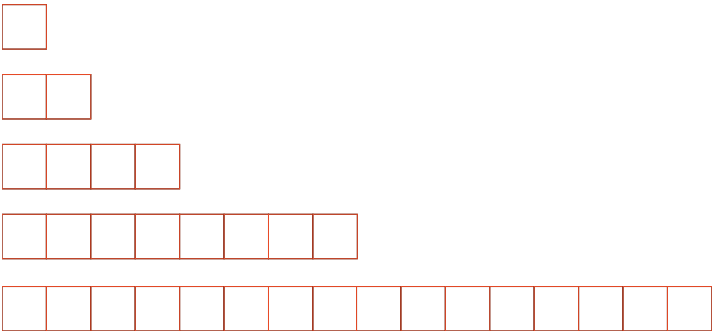
```
1 #include "Stack.h"
2
3 void Stack::push(T & t) {
4
5
6
7
8
9
10 }
11
12 T & Stack::pop() {
13
14
15
16
17
18
19 }
```

Resize Strategy – Details:

Strategy #1:



Strategy #2:



List Implementation #2: Array

```

Stack.cpp
1 #include "Stack.h"
2
3 template <class T>
4 void Stack::push(T & t) {
5     // If we are about to overflow, double the size of the array:
6     if (count_ + 1 == size_) {
7         size_ *= 2;
8         T * newArray = new T[size_];
9         for (unsigned i=0; i < count_; i++) { newArray[i] =
10 arr_[i]; }
11         delete arr_;
12         arr_ = newArray;
13     }
14
15     // Insert (push) the element into the array-backed stack:
16     arr[ count_++ ] = t;
17 }
18
19 template <class T>
20 T & Stack::pop() {
21     return arr[ --count_ ];
22 }

```

```

Stack.h
1 #ifndef STACK_H
2 #define STACK_H
3
4 template <class T>
5 class Stack {
6 public:
7     Stack();
8     Stack(const Stack &other);
9     ~Stack();
10    Stack& operator=(const Stack &other);
11
12    void push(T & t);
13    T & pop();
14    bool isEmpty() const;
15
16 private:
17     T * arr_;
18     unsigned size_, count_;
19
20 };
21
22 #endif

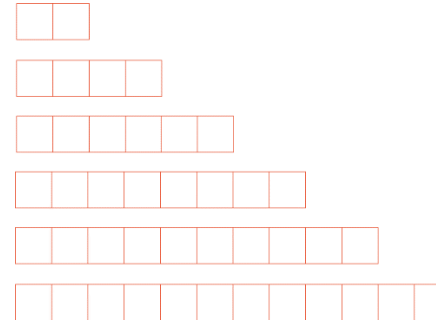
```

T* arr:

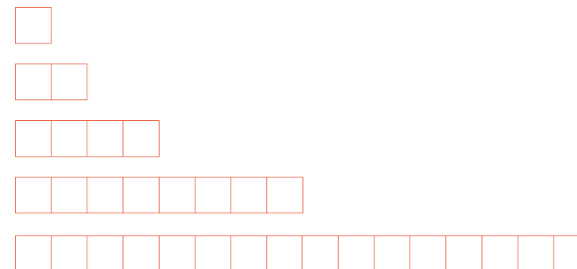
C	S	2	2	5		
[0]	[1]	[2]	[3]	[4]		

Resize Strategy – Details:

Strategy #1:



Strategy #2:



Three designs for data storage in data structures:

1. Not possible / T & data
2. T** arr / T* data
3. T* arr / T data

Implication of Design

1. Who manages the lifecycle of the data?
2. Is it possible to store a NULL as the data?
3. If the data is manipulated by user code while stored in our data structure, are the changes reflected within our data structure?
4. Is it possible to store literals?
5. Speed

	Storage by Reference	Storage by Pointer	Storage by Value
Lifecycle management of data?			
Possible to insert NULL?			
External data manipulation?			
Literal storage?			
Speed			

Queue.h	
1	#ifndef QUEUE_H
2	#define QUEUE_H
3	
4	template <class T>
5	class Queue {
6	public:
7	
8	
9	
10	
11	
12	private:
13	
14	};
15	
16	#endif

A queue is a: _____ data structure

...which stands for:

Why do we care about stacks and queues?

CS 225 – Things To Be Doing:

1. Quiz 2 on Thursday
2. MP2 is due next Monday; MP3 will be released soon
3. lab_gdb2 is due on Sunday.